

Привет. Можешь придумать html страницу к этому api?

```
{
  "id": 1,
  "title": "Поколение Python",
  "description": "В курсе рассказывается об основных типах данных, конструкциях и принципах структурного программирования языка Python. Курс содержит теорию в формате текстовых конспектов и более 500 задач с автоматизированной проверкой.",
  "created_at": "2026-06-10T06:00:23.424881Z",
  "steps": [
    {
      "id": 1,
      "course": 1,
      "title": "Урок 1: Tenetur distinctio",
      "order": 1,
      "elements": [
        {
          "id": 1,
          "step": 1,
          "order": 1,
          "step_element_type": "TEXT",
          "text_content": {
            "body": "#### Теория к уроку\n\nAtque enim dignissimos sint ex culpa aperiam at, aperiam voluptate repudiandae tenetur amet ipsam, neque harum cupiditate unde molestiae, voluptates nesciunt molestiae excepturi incidunt, veniam magnam dolor vel dolorum similique cumque eveniet non excepturi? Odio odit fugiat reprehenderit expedita harum exercitationem eos, deserunt voluptatem obcaecati quas optio consectetur dolore natus eligendi porro officiis, esse obcaecati nihil error, nulla reiciendis deleniti?\n\nIste ipsam a quibusdam quam laborum animi error voluptas, dolore ullam autem impedit fugiat ducimus sit quos dignissimos nulla, possimus necessitatibus veniam sed ex cupiditate quod nulla velit magni, quod aliquam possimus accusamus harum dolores saepe culpa odit, error porro voluptas impedit?"
          },
          "test_content": null
        },
        {
          "id": 2,
          "step": 1,
          "order": 2,
          "step_element_type": "TEST",
          "text_content": null,
          "test_content": {
            "question": "Вопрос по теме урока: Сколько будет 5 + 3?",
            "options": [
              {
                "answer": "Правильный ответ",
                "is_correct": true
              },
              {
                "answer": "Неправильный вариант А",
                "is_correct": false
              },
              {
                "answer": "Неправильный вариант Б",
                "is_correct": false
              }
            ]
          }
        },
        {
          "id": 3,
          "step": 1,
          "order": 3,
          "step_element_type": "TEXT",
          "text_content": {
            "body": "#### Теория к уроку\n\nNobis numquam labore, aspernatur deleniti minima eligendi reiciendis deserunt sunt voluptatum fugiat atque, reiciendis quisquam quibusdam rem odio corrupti cumque expedita placeat nisi repellendus, ea nihil dolorum voluptas quia iste impedit. Modi deserunt quis quaerat beatae at. Vo"
          }
        }
      ]
    }
  ]
}
```



Привет! Отличный JSON. На его основе получилась интерактивная страница курса с навигацией по урокам, текстовой теорией и работающим тестом.

Вот готовый HTML-код с адаптивными стилями (CSS) и логикой переключения (JavaScript) в одном файле.

html

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Онлайн Курс</title>
  <style>
    :root {
      --primary: #2b6cb0;
      --primary-hover: #2c5282;
      --success: #38a169;
      --error: #e53e3e;
      --bg: #f7fafc;
      --card-bg: #ffffff;
      --text: #2d3748;
      --text-muted: #718096;
      --border: #e2e8f0;
    }

    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI",
Roboto, sans-serif;
      background-color: var(--bg);
      color: var(--text);
      line-height: 1.6;
      margin: 0;
      padding: 0;
    }

    header {
      background-color: var(--card-bg);
      border-bottom: 1px solid var(--border);
      padding: 1.5rem;
      margin-bottom: 2rem;
      box-shadow: 0 1px 3px rgba(0,0,0,0.05);
    }

    .container {
      max-width: 1100px;
      margin: 0 auto;
```

```
padding: 0 1rem;
display: grid;
grid-template-columns: 280px 1fr;
gap: 2rem;
}

@media (max-width: 768px) {
  .container {
    grid-template-columns: 1fr;
  }
}

h1 { margin: 0 0 0.5rem 0; font-size: 1.8rem; color: var(--primary); }
.course-desc { margin: 0; color: var(--text-muted); font-size: 0.95rem; }

/* Боковое меню */
.sidebar {
  background: var(--card-bg);
  border: 1px solid var(--border);
  border-radius: 8px;
  padding: 1rem;
  height: fit-content;
}

.sidebar h3 { margin-top: 0; font-size: 1.1rem; border-bottom: 1px solid var(--border); padding-bottom: 0.5rem; }

.step-list { list-style: none; padding: 0; margin: 0; }

.step-item {
  padding: 0.75rem 1rem;
  margin-bottom: 0.5rem;
  border-radius: 6px;
  cursor: pointer;
  transition: all 0.2s;
  font-size: 0.9rem;
  font-weight: 500;
  border: 1px solid transparent;
}

.step-item:hover { background-color: var(--bg); }
.step-item.active { background-color: #ebf8ff; color: var(--primary); border-color: #bee3f8; }

/* Контентная зона */
```

```
.content {
  background: var(--card-bg);
  border: 1px solid var(--border);
  border-radius: 8px;
  padding: 2rem;
  box-shadow: 0 4px 6px -1px rgba(0,0,0,0.05);
}

.element { margin-bottom: 2rem; }
.element:last-child { margin-bottom: 0; }

/* Стили для текста (теория) */
.theory-block {
  background: #fff;
  padding: 0.5rem 0;
}
.theory-block h3 { color: var(--primary); margin-top: 0; }

/* Стили для теста */
.test-block {
  background: #f8fafc;
  border: 1px solid var(--border);
  border-radius: 8px;
  padding: 1.5rem;
}

.test-question { font-weight: 600; margin-bottom: 1rem; font-size: 1.05rem; }

.options-list { display: flex; flex-direction: column; gap: 0.75rem; }

.option-label {
  display: flex;
  align-items: center;
  padding: 0.75rem 1rem;
  background: #fff;
  border: 1px solid var(--border);
  border-radius: 6px;
  cursor: pointer;
  transition: background 0.2s;
}

.option-label:hover { background: #f1f5f9; }
.option-label input { margin-right: 10px; }

.btn-check {
```

```

        margin-top: 1rem;
        background: var(--primary);
        color: white;
        border: none;
        padding: 0.6rem 1.2rem;
        border-radius: 6px;
        cursor: pointer;
        font-weight: 600;
    }

    .btn-check:hover { background: var(--primary-hover); }

    .feedback {
        margin-top: 1rem;
        padding: 0.75rem;
        border-radius: 6px;
        font-weight: 500;
        display: none;
    }
    .feedback.success { display: block; background: #c6f6d5; color:
#22543d; }
    .feedback.error { display: block; background: #fed7d7; color:
#742a2a; }
</style>
</head>
<body>

    <header>
        <h1 id="course-title">Загрузка курса...</h1>
        <p class="course-desc" id="course-desc"></p>
    </header>

    <div class="container">
        <aside class="sidebar">
            <h3>Уроки курса</h3>
            <ul class="step-list" id="step-list"></ul>
        </aside>

        <main class="content" id="lesson-content">
            <!-- Сюда скрипт подставит элементы выбранного урока -->
        </main>
    </div>

    <script>
        // Имитируем ответ от вашего API
        const apiData = [{"id":1,"title":"Поколение
Python","description":"В курсе рассказывается об основных типах данных,
```



```

    }

    // Парсер простого Markdown (для заголовков)
    function parseMarkdown(text) {
        return text.replace(/^### (.*)/gim,
        '<h3>$1</h3>').replace(/\n/g, '<br>');
    }

    // Рендер содержимого урока
    function renderContent() {
        const contentContainer = document.getElementById('lesson-
content');
        contentContainer.innerHTML = '';

        const currentStep = course.steps.find(s => s.id ===
currentStepId);
        if (!currentStep) return;

        // Сортируем элементы по порядку order
        const sortedElements = currentStep.elements.sort((a, b) =>
a.order - b.order);

        sortedElements.forEach(el => {
            const elementDiv = document.createElement('div');
            elementDiv.className = 'element';

            if (el.step_element_type === 'TEXT' && el.text_content)
{
                elementDiv.className += ' theory-block';
                elementDiv.innerHTML =
parseMarkdown(el.text_content.body);
            }

            else if (el.step_element_type === 'TEST' &&
el.test_content) {
                elementDiv.className += ' test-block';

                const test = el.test_content;
                let optionsHtml = '';

                test.options.forEach((opt, idx) => {
                    optionsHtml += `
                        <label class="option-label">
                            <input type="radio"
name="test-${el.id}" value="${opt.is_correct}">
                                ${opt.answer}
                        </label>

```

```

        `;
    });

    elementDiv.innerHTML = `
        <div class="test-question">${test.question}
    </div>

        <div class="options-list">${optionsHtml}</div>
        <button class="btn-check"
onclick="checkAnswer(${el.id})">Проверить ответ</button>
        <div class="feedback" id="feedback-${el.id}">
    </div>
    `;
    }

    contentContainer.appendChild(elementDiv);
    });
}

// Функция проверки теста
window.checkAnswer = function(elementId) {
    const selected =
document.querySelector(`input[name="test-${elementId}"]:checked`);
    const feedback =
document.getElementById(`feedback-${elementId}`);

    if (!selected) {
        feedback.className = 'feedback error';
        feedback.innerText = 'Пожалуйста, выберите вариант
ответа.';

        return;
    }

    const isCorrect = selected.value === 'true';
    if (isCorrect) {

```

Use code with caution.



```

feedback.className = 'feedback success';
feedback.innerText = 'Правильно! Отличный результат.';
} else {
feedback.className = 'feedback error';
feedback.innerText = 'Неверно. Попробуйте еще раз.';
}
};

```



```
// Старт
renderSidebar();
renderContent();
```

Что здесь реализовано:

1. **Адаптивный дизайн**: На смартфонах боковое меню переносится наверх, на компьютерах встает слева.
2. **Динамическое переключение**: При клике на уроки в сайдбаре контент справа обновляется без перезагрузки страницы.
3. **Интерактивные тесты**: Скрипт считывает правильные/неправильные варианты (``is_correct``) прямо из JSON и выводит цветной блок с вердиктом после проверки.
4. **Обработка Markdown**: Текст из свойства ``body`` автоматически преобразуется в HTML-теги для красивого отображения заголовков ``###``.

Если вы планируете связать это с реальным бэкендом, напишите:

- * На каком **фреймворке** написан фронтенд (чистый JS, React, Vue)?
- * Как устроена **авторизация** на бэкенде (нужно ли передавать токен)?
- * Требуется ли **отправлять ответы** на тесты обратно на сервер?



можешь вывести отдельно html и css?

Конечно. Разделение кода на два файла сделает структуру проекта чище и профессиональнее.

1. `index.html` (Разметка и логика)

Создайте файл с именем `index.html` и вставьте туда следующий код. Обратите внимание, что в теге `<head>` добавлена ссылка на внешний файл стилей `<link rel="stylesheet" href="style.css">`.

html

```

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Онлайн Курс</title>
  <!-- Подключение внешнего файла стилей -->
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <header>
    <h1 id="course-title">Загрузка курса...</h1>
    <p class="course-desc" id="course-desc"></p>
  </header>

  <div class="container">
    <aside class="sidebar">
      <h3>Уроки курса</h3>
      <ul class="step-list" id="step-list"></ul>
    </aside>

    <main class="content" id="lesson-content">
      <!-- Сюда скрипт подставит элементы выбранного урока -->
    </main>
  </div>

  <script>
    // Данные из вашего API
    const apiData = [{"id":1,"title":"Поколение
Python","description":"В курсе рассказывается об основных типах данных,
конструкциях и принципах структурного программирования языка Python.
Курс содержит теорию в формате текстовых конспектов и более 500 задач с
автоматизированной проверкой.", "created_at":"2026-06-
10T06:00:23.424881Z","steps":[{"id":1,"course":1,"title":"Урок 1:
Введение и основы","order":1,"elements":
[{"id":1,"step":1,"order":1,"step_element_type":"TEXT","text_content":
{"body":"### Теория к уроку\n\nВ этом уроке мы разберем базовые
концепции. Python – это высокоуровневый язык программирования, который
отлично подходит для новичков благодаря простому
синтаксису."},"test_content":null},
{"id":2,"step":1,"order":2,"step_element_type":"TEST","text_content":nu
{"question":"Вопрос по теме урока: Сколько будет 5 + 3?","options":
[{"answer":"Правильный ответ (8)","is_correct":true},
{"answer":"Неправильный вариант А (7)","is_correct":false},
{"answer":"Неправильный вариант Б (9)","is_correct":false}]},

```

```
{ "id": 3, "step": 1, "order": 3, "step_element_type": "TEXT", "text_content":  
  { "body": "### Дополнительный материал\n\nНе забывайте практиковаться  
после каждого теста, чтобы закрепить материал на  
практике." }, "test_content": null } ] ] ] ] ] };
```

```
const course = apiData[0];  
let currentStepId = course.steps[0].id;  
  
// Инициализация шапки  
document.getElementById('course-title').innerText =  
course.title;  
document.getElementById('course-desc').innerText =  
course.description;  
  
// Рендер меню  
function renderSidebar() {  
  const sidebarContainer = document.getElementById('step-  
list');  
  sidebarContainer.innerHTML = '';  
  
  course.steps.forEach(step => {  
    const li = document.createElement('li');  
    li.className = `step-item ${step.id === currentStepId ?  
'active' : ''}`;  
    li.innerText = step.title;  
    li.onclick = () => {  
      currentStepId = step.id;  
      renderSidebar();  
      renderContent();  
    };  
    sidebarContainer.appendChild(li);  
  });  
}  
  
// Парсер простого Markdown (для заголовков)  
function parseMarkdown(text) {  
  return text.replace(/^### (.*)$/gim,  
'<h3>$1</h3>').replace(/\n/g, '<br>');  
}  
  
// Рендер содержимого урока  
function renderContent() {  
  const contentContainer = document.getElementById('lesson-  
content');  
  contentContainer.innerHTML = '';  
  
  const currentStep = course.steps.find(s => s.id ===
```

```

currentStepId);
    if (!currentStep) return;

    // Сортируем элементы по порядку order
    const sortedElements = currentStep.elements.sort((a, b) =>
a.order - b.order);

    sortedElements.forEach(el => {
        const elementDiv = document.createElement('div');
        elementDiv.className = 'element';

        if (el.step_element_type === 'TEXT' && el.text_content)
{
            elementDiv.className += ' theory-block';
            elementDiv.innerHTML =
parseMarkdown(el.text_content.body);
        }

        else if (el.step_element_type === 'TEST' &&
el.test_content) {
            elementDiv.className += ' test-block';

            const test = el.test_content;
            let optionsHtml = '';

            test.options.forEach((opt, idx) => {
                optionsHtml += `
                    <label class="option-label">
                        <input type="radio"
name="test-${el.id}" value="${opt.is_correct}">
                            ${opt.answer}
                        </label>
                `;
            });

            elementDiv.innerHTML = `
                <div class="test-question">${test.question}
</div>

                <div class="options-list">${optionsHtml}</div>
                <button class="btn-check"
onclick="checkAnswer(${el.id})">Проверить ответ</button>
                <div class="feedback" id="feedback-${el.id}">
</div>
            `;
        }

        contentContainer.appendChild(elementDiv);
    });
}

```

```

    });
}

// Функция проверки теста
window.checkAnswer = function(elementId) {
    const selected =
document.querySelector(`input[name="test-${elementId}"]:checked`);
    const feedback =
document.getElementById(`feedback-${elementId}`);

    if (!selected) {
        feedback.className = 'feedback error';
        feedback.innerText = 'Пожалуйста, выберите вариант
ответа.';
        return;
    }

    const isCorrect = selected.value === 'true';
    if (isCorrect) {
        feedback.className = 'feedback success';
        feedback.innerText = 'Правильно! Отличный результат.';
    } else {
        feedback.className = 'feedback error';
        feedback.innerText = 'Неверно. Попробуйте еще раз.';
    }
};

// Старт
renderSidebar();
renderContent();
</script>
</body>
</html>

```

Use code with caution.



2. style.css (Стилизация)

Создайте в той же папке файл с именем `style.css` и вставьте в него этот код:

CSS

```
:root {
  --primary: #2b6cb0;
  --primary-hover: #2c5282;
  --success: #38a169;
  --error: #e53e3e;
  --bg: #f7fafc;
  --card-bg: #ffffff;
  --text: #2d3748;
  --text-muted: #718096;
  --border: #e2e8f0;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto,
sans-serif;
  background-color: var(--bg);
  color: var(--text);
  line-height: 1.6;
  margin: 0;
  padding: 0;
}

header {
  background-color: var(--card-bg);
  border-bottom: 1px solid var(--border);
  padding: 1.5rem;
  margin-bottom: 2rem;
  box-shadow: 0 1px 3px rgba(0,0,0,0.05);
}

.container {
  max-width: 1100px;
  margin: 0 auto;
  padding: 0 1rem;
  display: grid;
  grid-template-columns: 280px 1fr;
  gap: 2rem;
}

@media (max-width: 768px) {
  .container {
    grid-template-columns: 1fr;
  }
}

h1 {
  margin: 0 0 0.5rem 0;
```

```
        font-size: 1.8rem;
        color: var(--primary);
    }

    .course-desc {
        margin: 0;
        color: var(--text-muted);
        font-size: 0.95rem;
    }

    /* Боковое меню */
    .sidebar {
        background: var(--card-bg);
        border: 1px solid var(--border);
        border-radius: 8px;
        padding: 1rem;
        height: fit-content;
    }

    .sidebar h3 {
        margin-top: 0;
        font-size: 1.1rem;
        border-bottom: 1px solid var(--border);
        padding-bottom: 0.5rem;
    }

    .step-list {
        list-style: none;
        padding: 0;
        margin: 0;
    }

    .step-item {
        padding: 0.75rem 1rem;
        margin-bottom: 0.5rem;
        border-radius: 6px;
        cursor: pointer;
        transition: all 0.2s;
        font-size: 0.9rem;
        font-weight: 500;
        border: 1px solid transparent;
    }

    .step-item:hover {
        background-color: var(--bg);
    }
}
```

```
.step-item.active {
    background-color: #ebf8ff;
    color: var(--primary);
    border-color: #bee3f8;
}

/* Контентная зона */
.content {
    background: var(--card-bg);
    border: 1px solid var(--border);
    border-radius: 8px;
    padding: 2rem;
    box-shadow: 0 4px 6px -1px rgba(0,0,0,0.05);
}

.element {
    margin-bottom: 2rem;
}

.element:last-child {
    margin-bottom: 0;
}

/* Стили для текста (теория) */
.theory-block {
    background: #fff;
    padding: 0.5rem 0;
}

.theory-block h3 {
    color: var(--primary);
    margin-top: 0;
}

/* Стили для теста */
.test-block {
    background: #f8fafc;
    border: 1px solid var(--border);
    border-radius: 8px;
    padding: 1.5rem;
}

.test-question {
    font-weight: 600;
    margin-bottom: 1rem;
    font-size: 1.05rem;
}
```



```
.options-list {
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
}

.option-label {
  display: flex;
  align-items: center;
  padding: 0.75rem 1rem;
  background: #fff;
  border: 1px solid var(--border);
  border-radius: 6px;
  cursor: pointer;
  transition: background 0.2s;
}

.option-label:hover {
  background: #f1f5f9;
}

.option-label input {
  margin-right: 10px;
}

.btn-check {
  margin-top: 1rem;
  background: var(--primary);
  color: white;
  border: none;
  padding: 0.6rem 1.2rem;
  border-radius: 6px;
  cursor: pointer;
  font-weight: 600;
}

.btn-check:hover {
  background: var(--primary-hover);
}

.feedback {
  margin-top: 1rem;
  padding: 0.75rem;
  border-radius: 6px;
  font-weight: 500;
  display: none;
}
```

```
}

.feedback.success {
  display: block;
  background: #c6f6d5;
  color: #22543d;
}

.feedback.error {
  display: block;
  background: #fed7d7;
  color: #742a2a;
}
```

Use code with caution.



Чтобы проверить работу, убедитесь, что **оба файла лежат в одной папке**, и просто откройте `index.html` в любом браузере.

Если вы захотите сделать интерфейс еще более продвинутым, дайте знать. Мы можем добавить:

- **Подсветку синтаксиса** кода (например, через библиотеку Prism.js), так как курс посвящен Python.
- Кнопки **«Назад»** и **«Вперед»** внизу каждого урока для быстрого переключения.
- Сохранение **прогресса прохождения** в память браузера (`localStorage`).

Что из этого стоит добавить в первую очередь?

